- **Prérequis :**

```
PS C:\Windows\system32> docker --version
Docker version 29.4.0, build 9d7ad9f
PS C:\Windows\system32> minikube version
minikube version: v1.38.1
commit: c93a4cb9311efc66b90d33ea03f75f2c4120e9b0
PS C:\Windows\system32> kubectl version --client
Client Version: v1.35.0
Kustomize Version: v5.7.1
```

- **Démarrage du cluster Kubernetes local :**

```
PS C:\Windows\system32> minikube start --driver=docker
* minikube v1.38.1 sur Microsoft Windows 11 Pro 23H2
* Utilisation du pilote docker basé sur la configuration de l'utilisateur

X Fermeture en raison de PROVIDER_DOCKER_VERSION_EXIT_1 : "docker version --format <no value>-<no value>:<no value>" exit status 1: Error response from daemon: Docker Deskt
op is unable to start
* Documentation: https://minikube.sigs.k8s.io/docs/drivers/docker/
```

Le cluster ne démarre pas car WSL n'est pas à jour, je l'ai donc mis à jour via le .msi de ce dépôt github : https://github.com/microsoft/WSL/releases

```
PS C:\Windows\system32> minikube start --driver=docker
* minikube v1.38.1 sur Microsoft Windows 11 Pro 23H2
* Utilisation du pilote docker basé sur la configuration de l'utilisateur
! Starting v1.39.0, minikube will default to "containerd" container runtime. See #21973 for more info.
* Utilisation du pilote Docker Desktop avec le privilège root
* Démarrage du nœud "minikube" primary control-plane dans le cluster "minikube"
* Extraction de l'image de base v0.0.50...
    > gcr.io/k8s-minikube/kicbase...:  519.58 MiB / 519.58 MiB  100.00% 8.48 Mi
* Création de docker container (CPU=2, Memory=4000Mo) ...
* Préparation de Kubernetes v1.35.1 sur Docker 29.2.1...
* Configuration de bridge CNI (Container Networking Interface)...
* Vérification des composants Kubernetes...
  - Utilisation de l'image gcr.io/k8s-minikube/storage-provisioner:v5
* Modules activés: storage-provisioner, default-storageclass
* Terminé ! kubectl est maintenant configuré pour utiliser "minikube" cluster et espace de noms "default" par défaut.
```

On peut voir que le cluster démarre une fois la mise à jour effectué.

```
PS C:\Windows\system32> kubectl get nodes
NAME       STATUS   ROLES           AGE   VERSION
minikube   Ready    control-plane   9s    v1.35.1
```

Vérification de la création du noeud.

- **Déploiement d'une application distribuée**

```
PS C:\Windows\system32> kubectl create deployment web --image=nginx
deployment.apps/web created
```

```
PS C:\Windows\system32> kubectl get deployments
NAME   READY   UP-TO-DATE   AVAILABLE   AGE
web    3/3     3            3           7m52s
```

Vérification de la présence du déploiement.

```
PS C:\Windows\system32> kubectl get pods
NAME                   READY   STATUS    RESTARTS   AGE
web-68d995574f-gswfz   1/1     Running   0          10s
```

Vérification des instances actives. (1 seul pour le moment)

- **Mise à l'échelle (scaling)**

```
PS C:\Windows\system32> kubectl scale deployment web --replicas=3
deployment.apps/web scaled
```

```
PS C:\Windows\system32> kubectl get pods
NAME                    READY   STATUS    RESTARTS   AGE
web-68d995574f-4sdn4    1/1     Running   0          8m26s
web-68d995574f-8qb4h    1/1     Running   0          85s
web-68d995574f-q7dt9    1/1     Running   0          85s
```

À la suite de la création de ces 3 instances, on peut voir que la commande affiche maintenant 3 lignes avec donc nos 3 instances opérationnelles.

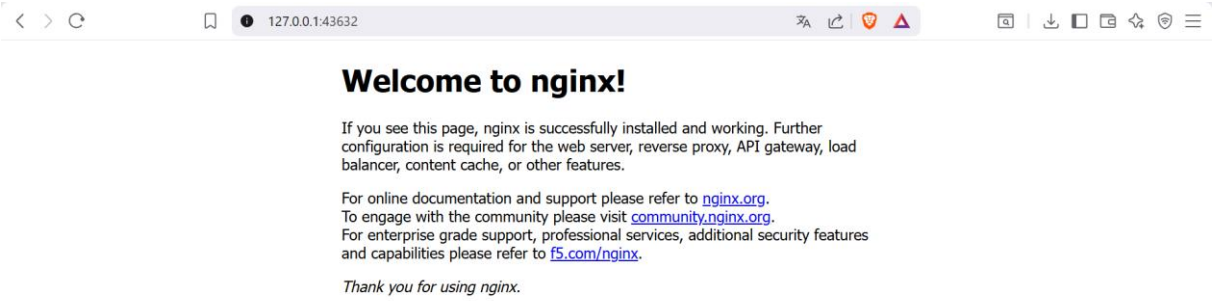
- **Exposition du service**

```
PS C:\Windows\system32> kubectl expose deployment web --type=NodePort --port=80
service/web exposed
```

```
PS C:\Windows\system32> minikube service web --url
http://127.0.0.1:43632
! Comme vous utilisez un pilote Docker sur windows, le terminal doit être ouvert pour l'exécuter.
```

Cette commande nous permet d'obtenir l'URL à exploiter pour accéder à notre serveur WEB.

On se connecte à l'URL qui nous est fourni : **http://127.0.0.1:43632**



Voici la page web affiché lorsqu'on se rend sur l'URL.

- **Simulation d'une panne**

```
PS C:\Windows\system32> kubectl delete pod web-68d995574f-4sdn4
pod "web-68d995574f-4sdn4" deleted from default namespace
```

```
PS C:\Windows\system32> kubectl get pods
NAME                    READY   STATUS    RESTARTS   AGE
web-68d995574f-8qb4h    1/1     Running   0          13m
web-68d995574f-pwrd2    1/1     Running   0          79s
web-68d995574f-q7dt9    1/1     Running   0          13m
```

*On voit qu'une instance a été recréé à la place de notre ancienne instance : web-68d995574f-**4sdn4***

- **Bonus (Ajouter le nombre de replicas)**

```
PS C:\Windows\system32> kubectl scale deployment web --replicas=5
deployment.apps/web scaled
```

```
PS C:\Windows\system32> kubectl get pods
NAME                    READY   STATUS    RESTARTS   AGE
web-68d995574f-7zrvs    1/1     Running   0          42s
web-68d995574f-8qb4h    1/1     Running   0          26m
web-68d995574f-pwrd2    1/1     Running   0          14m
web-68d995574f-q7dt9    1/1     Running   0          26m
web-68d995574f-r9ch9    1/1     Running   0          42s
```

On observe bien qu'il y a maintenant 5 instances actives.

- **Questions**

1. Quelle est la différence entre un système centralisé et distribué ?
2. Pourquoi utiliser plusieurs instances d'une application ?
3. Que se passe-t-il si un pod tombe en panne ?
4. Qu'est-ce que la tolérance aux fautes ?
5. Kubernetes garantit-il la haute disponibilité ?
6. Quel est le rôle du load balancing ?

1. Pour comprendre cette différence, définissons d'abord ce qu'est un **nœud** : il s'agit d'une unité de calcul (un ordinateur ou un serveur) capable d'exécuter un programme.

- **Système centralisé :** Toute l'intelligence et les données reposent sur un seul nœud principal. Si ce nœud tombe en panne, l'intégralité du service s'arrête. C'est le principe du "point de défaillance unique" (*Single Point of Failure*).
- **Système distribué :** Les tâches et les données sont réparties sur plusieurs nœuds qui communiquent entre eux via un réseau pour atteindre un objectif commun. Si l'un des nœuds tombe, les autres continuent de fonctionner, garantissant la continuité du service.

2. Une **instance** est une copie active et autonome d'un programme en cours d'exécution. On en utilise plusieurs pour deux raisons majeures :

- **La répartition de la charge (Scalabilité) :** Si des milliers d'utilisateurs se connectent en même temps, une seule instance pourrait être saturée. En multipliant les instances, on divise le travail.
- **La redondance :** Avoir plusieurs copies permet de s'assurer que si une instance plante (à cause d'un bug ou d'une saturation), les autres copies sont déjà prêtes à prendre le relais sans que l'utilisateur ne s'en aperçoive.

3. Dans l'univers Kubernetes, un **Pod** est la plus petite unité d'exécution. C'est une sorte d'enveloppe qui contient ton application (le conteneur).

Si un Pod tombe en panne, Kubernetes active son mécanisme d'**auto-guérison** (*self-healing*). L'orchestrateur (le cerveau du système) compare en permanence l'**état actuel** (le Pod est mort) avec l'**état désiré** (le développeur a demandé qu'un Pod tourne). Constatant l'écart, Kubernetes va automatiquement détruire le Pod défectueux et en recréer un nouveau sur un nœud disponible pour rétablir le service.

4. La **tolérance aux fautes** est la capacité d'un système à continuer de fonctionner correctement, éventuellement en mode dégradé, malgré la panne d'un ou plusieurs de ses composants (matériels ou logiciels).

Contrairement à un système classique qui s'arrête net lors d'une erreur, un système tolérant aux fautes est conçu pour anticiper les pannes. Il utilise pour cela la **redondance** (avoir des remplaçants prêts) et l'**isolation** (faire en sorte qu'une panne sur un composant ne se propage pas aux autres).

5. La **haute disponibilité (HA)** est une mesure de performance qui vise à garantir qu'un service est accessible 99,9% du temps (ou plus).

Kubernetes ne la "garantit" pas par magie, mais il fournit tous les outils pour la **faciliter**. Pour obtenir une véritable haute disponibilité, l'administrateur doit configurer Kubernetes de manière à ce que les nœuds soient répartis sur plusieurs serveurs physiques ou zones géographiques différentes. Si Kubernetes lui-même est installé sur un seul serveur qui brûle, il n'y a plus de haute disponibilité. C'est donc un outil puissant, mais son efficacité dépend de la configuration de l'infrastructure.

6. Le **load balancing** (ou équilibrage de charge) est un mécanisme qui agit comme un "agent de circulation" pour le trafic réseau.

Son rôle est de distribuer les requêtes entrantes des utilisateurs de manière équitable entre toutes les instances disponibles de l'application. Cela évite qu'une seule instance ne reçoive tout le trafic et ne s'effondre sous le poids des demandes, pendant que les autres restent inactives. Il s'assure également de ne jamais envoyer de trafic vers une instance qu'il a détectée comme étant en panne.